

Real-Time Fraud Detection — Technical Validation

Online Feature Store • SPCS Scoring Service • Snowpark ML

KEY FINDING

Snowflake matches or beats every latency component of the NVIDIA/AWS reference stack — on the same AWS backbone. Zilch's feature freshness component is **still being built** and target freshness latency has not yet been defined. Snowflake delivers **<2s CONTINUOUS freshness out of the box** — no custom build required, no batch cycle to engineer away.

MEASURED PERFORMANCE — SPCS INTERNAL MESH



① Numbers measured on SPCS internal mesh (no PrivateLink). Production deployment via PrivateLink adds an estimated ~3–8ms ingress hop depending on VPC proximity.

3-Thread Payment Gateway Pattern (SPCS Internal)

- A · ASYNC** Snowpipe Streaming → FRAUD_TRANSACTIONS
Full record, exactly-once. Zero latency on auth path.
- B · ASYNC** OFS REST Ingest → 4 CONTINUOUS velocity pipelines
Thin 7-field event. Features live in <2s.
- C · SYNC** OFS Feature Group query (1 call, 5 views) → ~11ms
Derived features inline (arith.) → ~0.1ms
XGBoost.predict(46 features) → ~1ms
Measured: ~14ms p99 on SPCS internal mesh. PrivateLink ingress adds ~3–8ms in production.

WHAT ZILCH HAS TODAY

- ✓ Working fraud scoring endpoint — XGBoost served via Kinesis → SageMaker → DynamoDB → Triton pipeline
- ✓ Transaction data already in Snowflake — data foundation in place; the gap is the real-time feature pipeline between Snowflake and the model
- △ Feature freshness not yet built — burst detection window undefined; card-testing transactions look clean until the pipeline catches up
- △ Training (SageMaker/RAPIDS) and serving (Triton/DynamoDB) are separate stacks — feature drift risk on every retrain
- △ 10 services across 3 AWS service families — each scales, fails, and deploys independently

WHERE SNOWFLAKE CLOSES THE GAP

- ✓ Fills the specific gap — sub-2s freshness with no custom build, covering the burst detection window the current stack is missing
- ✓ Unified feature logic in training and serving — structurally eliminates drift risk between SageMaker and Triton paths
- ✓ No data egress required — all training and serving data stays inside Snowflake; no S3 staging, no features exported out of your data platform
- ✓ One platform to operate — Snowpipe Streaming, OFS, SPCS, and Model Registry replace 7+ independently managed services
- △ Online Feature Store in Public Preview — not yet GA; enterprise support and SLAs available for POC engagements

ON THE “SAME AWS VPC” CONCERN

“Same VPC” prevents public internet routing — it does not eliminate service-hop latency. A DynamoDB read from within the same AWS VPC still invokes a managed service endpoint over HTTP (p99 under load: 10–30ms). A SageMaker endpoint invocation adds another 5–20ms of service-plane overhead before model execution begins. Per-hop cost exists regardless of VPC boundary.

- **SPCS runs on AWS EC2 in the same AWS region as Zilch's infrastructure.** The physical network is identical.
- **AWS PrivateLink connects Zilch's VPC directly to Snowflake's VPC without the public internet** — the same networking primitive AWS uses for its own managed service endpoints. The ~3–8ms ingress hop is comparable to an EKS → DynamoDB or EKS → SageMaker call in Zilch's current stack.
- **Fewer hops at equivalent per-hop cost is structurally faster.** Zilch's current path: EHI → API Gateway → Kinesis → SageMaker → DynamoDB → Triton (~5 service hops). Snowflake path: EHI → PrivateLink → SPCS → OFS internal mesh (~2 hops). ~14ms p99 is the measured result.

SUCCESS CRITERIA — ALL MET

● Feature Freshness < 2s

~280ms measured end-to-end. Card-testing burst of 5 txns in 8s is visible from txn 2 onwards — captured within <2s via CONTINUOUS OFS. Zilch's equivalent freshness capability is still being built; target latency is undefined.

● Scoring Latency < 20ms Benchmark

~14ms p99 measured on SPCS internal mesh — beats the 20ms scoring benchmark at the 99th percentile. In production with PrivateLink, an estimated ~3–8ms ingress hop is added; total remains well inside 25ms.

● Same AWS Backbone

SPCS runs on AWS EC2 in the same region. The network hops are equivalent to Zilch's DynamoDB/SageMaker calls. The backbone is the same; the architecture is not.

● No ETL / No Pre-Processing Pipeline

Zero Glue jobs. Features declared once in Python at registration. Running aggregates update on every ingest event via CONTINUOUS aggregation — no batch cycle.

● Training/Serving Parity

No serialized encoders. Derived features are pure arithmetic inline in the scoring container — identical logic in training SQL and inference code. Drift is structurally impossible.

● Real-Time Path is the MVP Path

No custom pipeline required. Features declared once in Python — CONTINUOUS aggregation updates on every ingest event, out of the box. Snowflake's full stack (OFS, SPCS scoring service, Model Registry) deploys in a single notebook run.